

The terminal symbols ‘#’ and ‘\$\$’ used in a few of the states are auxiliary symbols introduced by the `fsyacc` parser generator to properly handle the start and end of the parsing process; we shall ignore them here.

3.6.5 Exercising the Parser Automaton

The parser generated by e.g. `fsyacc` looks like a finite automaton, but instead of just a single current state, it has a stack containing states and grammar symbols.

If there is an automaton state such as #20 on top of the stack, then that state’s action table and the next input symbol determines the action. As explained above, the action may be “shift” or “reduce”.

For example, if state 19 is on the stack top and the next input symbol is `*`, then according to the action table of state 19 (shown above) the action is shift 21 which means that `*` is removed from the input and pushed on the stack together with state #21.

If again state 19 is on the stack top but the next input symbol is EOF, then the action is to reduce by this grammar rule from the parser specification:

`Expr ::= Expr PLUS Expr`

The reduce action uses the grammar rule “in reverse”, to remove the grammar symbols `Expr PLUS Expr` from the stack, and push `Expr` instead. After a reduce action, the state below the new stack top symbol `Expr` (for instance, state 0) is inspected for a suitable goto rule (for instance, `goto Expr: 2`), and the new state 2 is pushed on the stack.

For a complete parsing example, consider the parser states traversed during parsing of the string `x + 52 * wk EOF`:

Input	Parse stack (top on right)	Action
x+52*wk&	#0	shift #4
+52*wk&	#0 x #4	reduce by B
+52*wk&	#0 Expr	goto #2
+52*wk&	#0 Expr #2	shift #22
52*wk&	#0 Expr #2 + #22	shift #5
*wk&	#0 Expr #2 + #22 52 #5	reduce by C
*wk&	#0 Expr #2 + #22 Expr	goto #19
*wk&	#0 Expr #2 + #22 Expr #19	shift #21
wk&	#0 Expr #2 + #22 Expr #19 * #21	shift #4
&	#0 Expr #2 + #22 Expr #19 * #21 wk #4	reduce by B
&	#0 Expr #2 + #22 Expr #19 * #21 Expr	goto #18
&	#0 Expr #2 + #22 Expr #19 * #21 Expr #18	reduce by G
&	#0 Expr #2 + #22 Expr	goto #19
&	#0 Expr #2 + #22 Expr #19	reduce by H

&	#0 Expr	goto #2
&	#0 Expr #2	shift 3
	#0 Expr #2 & #3	reduce by A
	#0 Main	goto #1
	#0 Main #1	accept

The numbers #0, #1, ...are parser state numbers from the ExprPar.fsyacc.output file, the symbol & is here used to represent end-of-file (EOF), and the letters A, B, and so on, indicate which grammar rule is used (backwards) in a reduce step:

Main ::= Expr EOF	rule A
Expr ::= NAME	rule B
CSTINT	rule C
MINUS CSTINT	rule D
LPAR Expr RPAR	rule E
LET NAME EQ Expr IN Expr END	rule F
Expr TIMES Expr	rule G
Expr PLUS Expr	rule H
Expr MINUS Expr	rule I

When the parser performs a reduction using a given rule, it evaluates the semantic action associated with that rule. It also pushes the result of the semantic action on the stack, but this is not shown above. Hence the \$1, \$2, and so on in a semantic action refers to the value, on the stack, associated with a given terminal or nonterminal symbol.

3.6.6 Shift/Reduce Conflicts

Studying the ExprPar.fsyacc.output file is useful if there are *shift/reduce conflicts* or *reduce/reduce conflicts* in the generated parser. Such conflicts arise because the grammar is ambiguous: some string may be derived in more than one way.

For instance, if we remove the precedence and associativity declarations (`%left`) from the tokens PLUS, MINUS and TIMES in the ExprPar.fsy parser specification, then there will be shift/reduce conflicts in the parser.

Then fsyacc will produce a conflict message like this on the console:

```
state 19: shift/reduce error on PLUS
state 19: shift/reduce error on TIMES
state 19: shift/reduce error on MINUS
```

and now state 19 looks like this:

```
state 19:
  items:
    Expr -> Expr . 'TIMES' Expr
    Expr -> Expr . 'PLUS' Expr
```